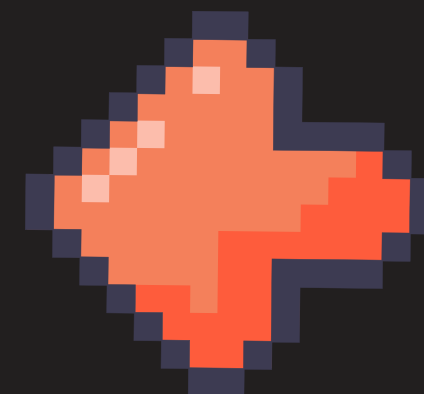
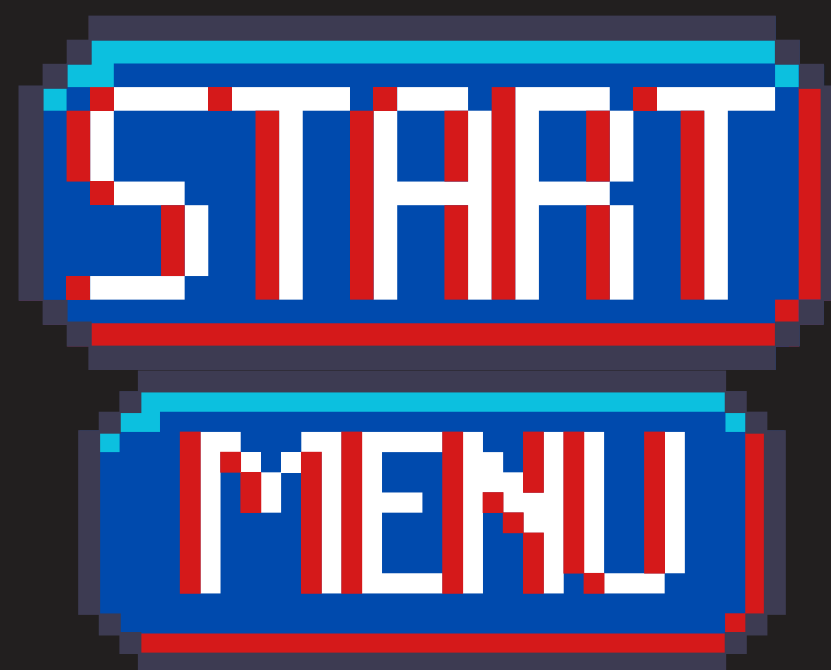


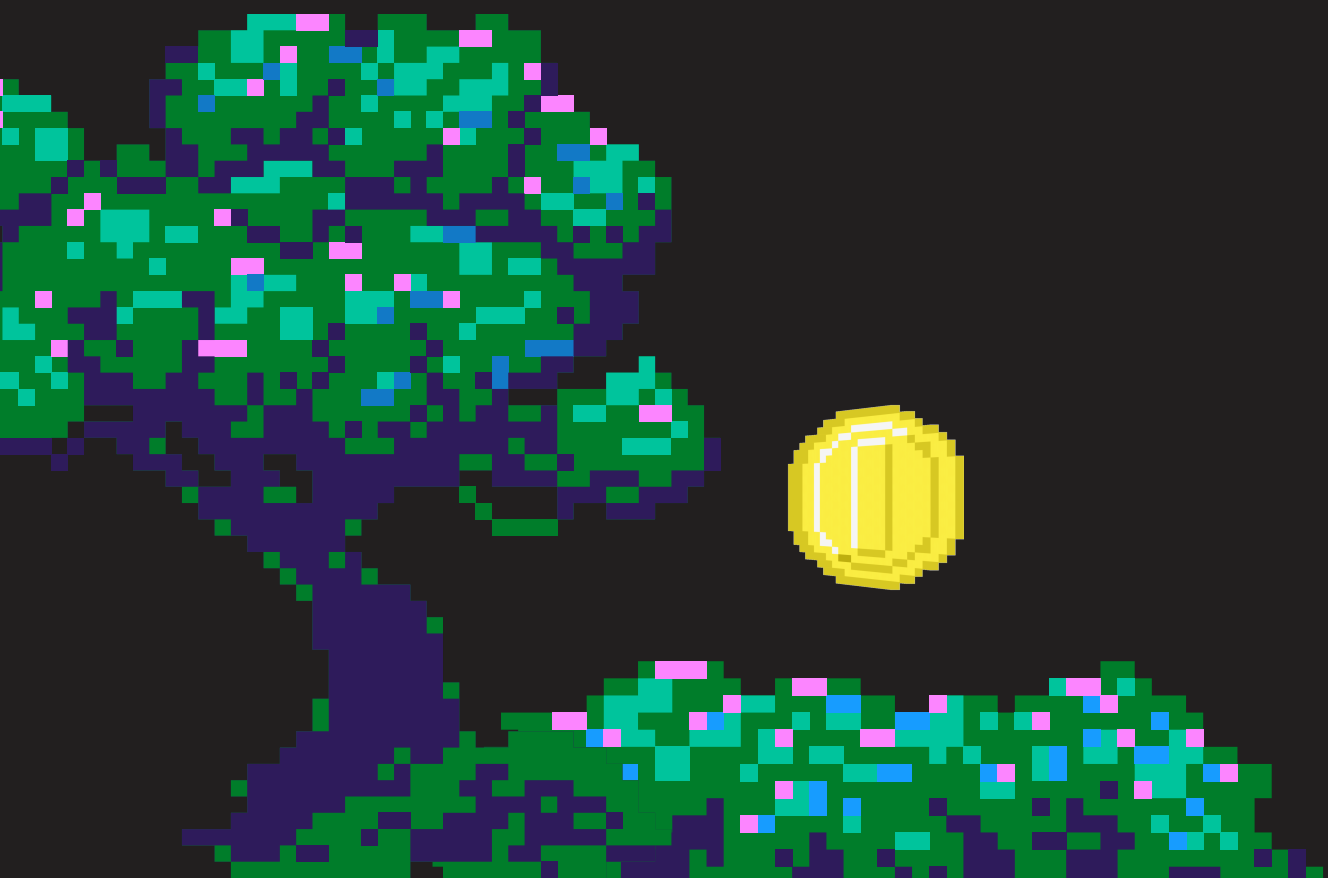
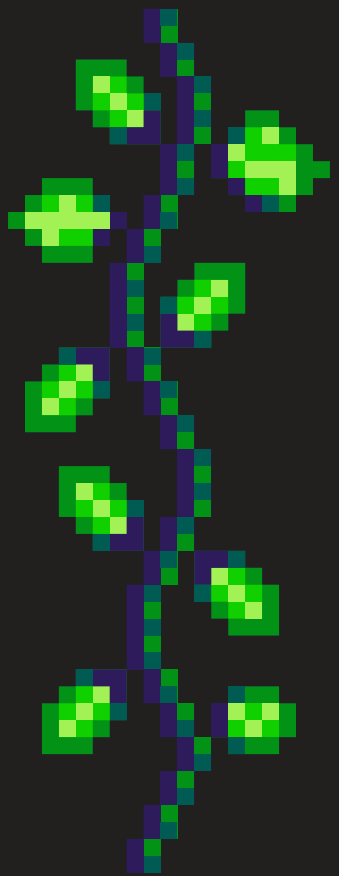
INTRODUÇÃO AO PYGAME



INTRODUÇÃO

→ Como o pygame funciona?

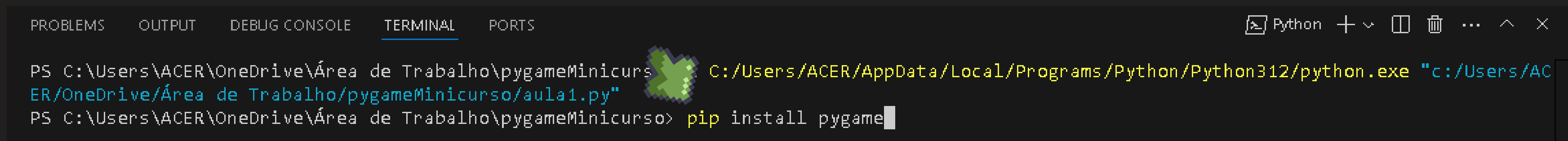
- Consiste em uma biblioteca repleta de módulos e funções que auxiliam na criação de um jogo
- Não é uma engine em si tal qual a Unity ou a Unreal



INSTALANDO A BIBLIOTECA

→ pip install pygame

- O primeiro passo será abrir o terminal do Vscode ou da sua IDE de preferência e executar o comando “pip install pygame”



The screenshot shows a VS Code terminal window with the following content:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + - [ ] [X] ... ^ X
```

PS C:\Users\ACER\OneDrive\Área de Trabalho\pygameMinicurs C:/Users/ACER/AppData/Local/Programs/Python/Python312/python.exe "c:/Users/ACER/OneDrive/Área de Trabalho/pygameMinicurso/aula1.py"

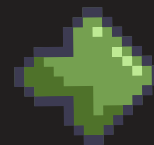
PS C:\Users\ACER\OneDrive\Área de Trabalho\pygameMinicurso> pip install pygame

criação de uma janela



→ A estrutura do loop base

- Todo jogo é formado por um loop (while ou for) que tem o objetivo de gerar atualizações na tela e buscar por inputs do usuário.
- Primeiramente iniciaremos os módulos da biblioteca por meio do `pygame.init()`
- Função `set_mode()` criará um objeto do tipo `Pygame.surface`
- `pygame.display.set_caption()`
- Verificação de eventos
- Clock



O loop ficará:

```
8  clock = pygame.time.Clock()
9
10 #Loop do jogo
11 while running:
12     #Checando por eventos
13     for event in pygame.event.get():
14         #Verificando tipo do evento
15         if event.type == pygame.QUIT:
16             pygame.quit()
17             exit()
18
19     clock.tick(60)
```



IMAGENS

→ Display surface e surface

- Display surface é a janela do jogo, onde todos os elementos do jogo aparecerão (Única e sempre visível)
- Uma surface é uma imagem importada, um texto ou uma figura preenchida por alguma cor

→ Criação de figura preenchida por cor

```
1 surface_teste = pygame.Surface((100,200))  
2 surface_teste.fill("Red")
```

- Aqui criamos uma surface e a preenchemos com a cor vermelha, dentro do loop usando .blit na display surface, iremos por essa outra surface por cima dela

IMAGENS

→ Imagem importada

- Com `pygame.image.load("path da imagem")` conseguimos criar surfaces com imagens específicas, como por exemplo para fazermos um fundo

```
1 image = pygame.image.load('ImageMagick.png')
```

CRIANDO TEXTO

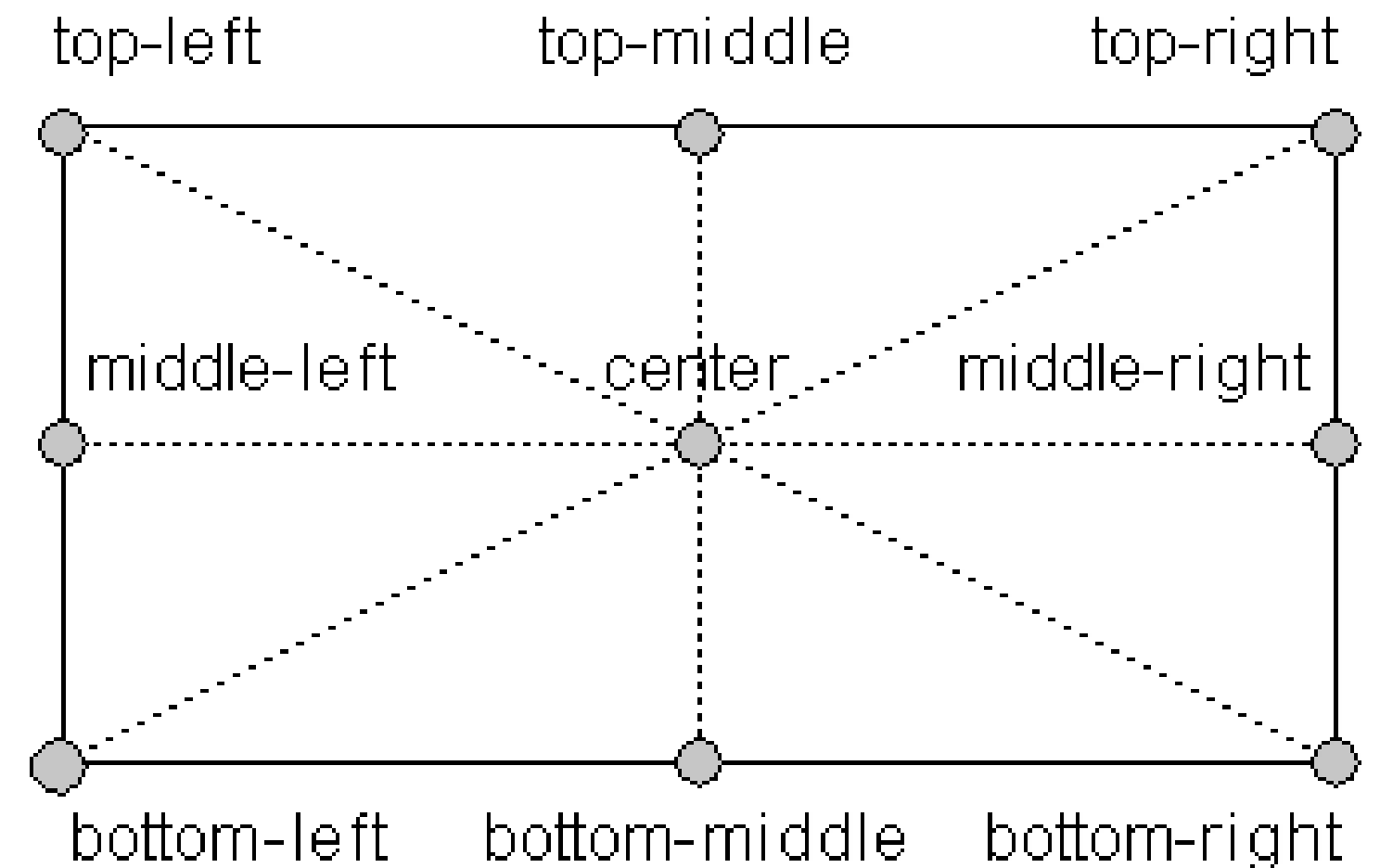
- Para colocar um texto na tela, precisamos criar uma superfície com esse texto, além de escolhermos o tamanho e a fonte do mesmo.

```
1 fonte = pygame.font.Font(None, 50)
2 texto_surface = fonte.render('texto', False, 'Green')
```

RETÂNGULOS

- Por meio da classe `pygame.Rect` podemos definir a posição, altura e largura de um retângulo na tela. Podemos acessar alguns pontos específicos do objeto por meio de algumas funções.
- Com Retângulos, conseguimos ter um controle melhor das superfícies no nosso código, permitindo assim, um controle melhor de suas coordenadas:

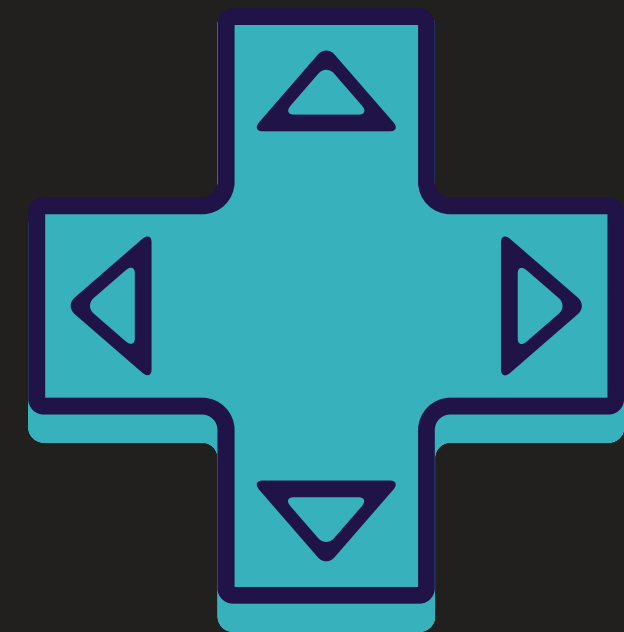
```
1 #Surface e retângulo do Fundo
2 fundo = pygame.image.load("sprites/fundo.png")
3 fundo_rect = fundo.get_rect(topleft = (0,0))
4 tela.blit(fundo, fundo_rect)
```



MOVIMENTAÇÃO E COLISÕES

→ Movimentações:

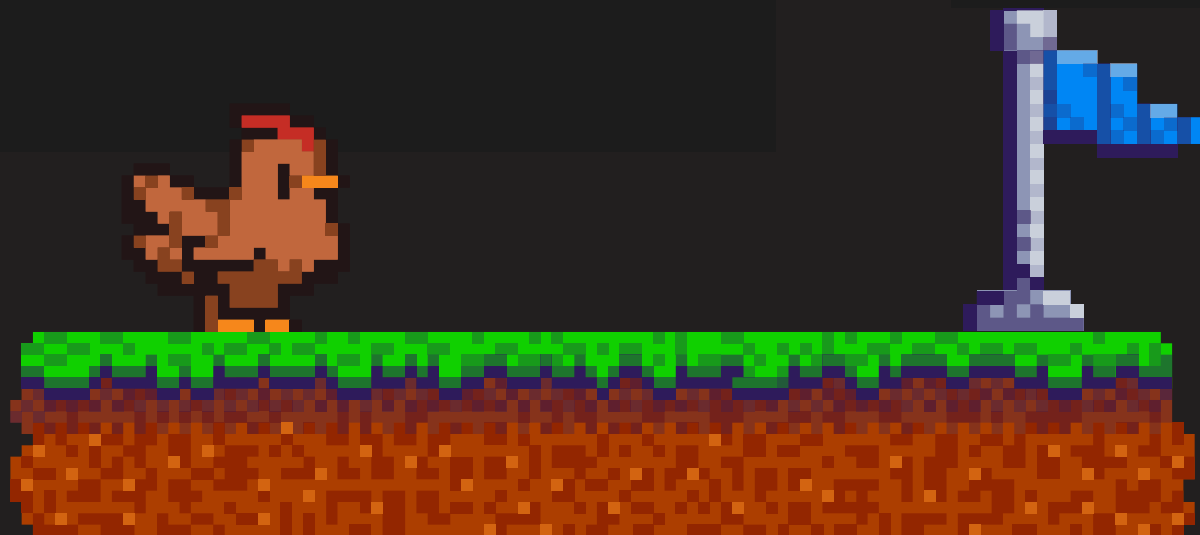
- Existem dois tipos de movimentação:
 - as que são movidas pelo próprio jogo
 - as que o jogador controla
- Como estamos rodando um jogo dentro de um laço de repetição, podemos alterar a posição de um objeto a cada leitura do laço:



```
1 player_surface = pygame.image.load("seu_png.png").convert_alpha()
2 player_rect = player_surface.get_rect(center = (300,300))
3
4 #dentro do while:
5 tela.fill('Black')
6 tela.blit(player_surface, player_rect)
7 player_rect.x += 1
```

Observação:

```
1 if player_rect.left >= 600:
2     player_rect.right = 0
```



MOVIMENTAÇÃO E COLISÕES

→ Colisões:

- Com a movimentação pronta, podemos determinar o que acontecerá com nossos objetos quando interagirem uns com os outros.
- Assim como usamos uma estrutura condicional (if) para checar se os objetos “saem” da tela, podemos usa-los para checar se os objetos se colidem com a função “.collidect()”:

```
1 tela.fill('Black')
2 tela.blit(player_surface, player_rect)
3 tela.blit(obstacle_surface, obstacle_rect)
4
5 if player_rect.collidect(obstacle_rect):
6     #score += 1, lifes -= 1....
```



CONTROLES

- Agora que sabemos como movimentar objetos dentro do jogo, podemos fazer também uma movimentação baseada nos inputs do teclado e mouse do usuário:

```
1  if event.type == pygame.KEYDOWN:  
2      if event.key == pygame.K_UP:  
3          player_rect.y -= 1  
4  
5  if event.type == pygame.KEYUP:  
6      if event.key == pygame.K_UP:  
7          print("soltou a tecla")
```

